B 3.1.1 Fundamentals of OOP

B 3.1.1 Evaluate the fundamentals of OOP.

OOP models real-world entities using classes, objects, inheritance, encapsulation, and polymorphism. This section evaluates the benefits and drawbacks of OOP in different programming contexts.

Learning Statement:

- Model real-world entities using OOP concepts: classes, objects, inheritance, encapsulation, polymorphism
- The advantages and disadvantages of using OOP in various programming scenarios

Learning Objectives:

By the end of this lesson, students will be able to:

- Define the fundamental concepts of Object-Oriented Programming (OOP): classes, objects, attributes, behaviours (methods), inheritance, encapsulation, and polymorphism.
- Explain how real-world entities can be modelled using these OOP concepts.
- Identify at least three advantages and three disadvantages of using OOP in software development.
- Discuss scenarios where OOP would be a suitable programming paradigm and scenarios where it might not be.

Relevant ATL Skills:

- **Thinking Skills:**
  - **Conceptual understanding:** Understanding concepts such as classes, objects, inheritance, encapsulation, and polymorphism.
  - **Critical thinking:** Analysing the advantages and disadvantages of OOP.
  - **Transfer skills:** Applying the concepts of OOP to model real-world entities.
- **Communication Skills:**
  - **Expressing ideas:** Articulating their understanding of OOP concepts and their application.

    - **Active listening:** Engaging in discussions and understanding different perspectives on the advantages and disadvantages of OOP.
- **Collaboration Skills:**
    - Working in pairs or small groups to discuss examples and complete worksheet activities.

Lesson Resources

Presentation

Link to presentation

eBook

The eBook covers all the material on the presentation in detail and also contains a multiple choice quiz for students to test their understanding.

B 3.1.1 Fundamentals of OOP

Other resources

B 3.1.1 Worksheet

B 3.1.1 Worksheet answers

Lesson Planning

Time	Activity	Content Covered	Covered Slide(s)
0-5 mins	Introduction and Setting the Scene	Briefly introduce OOP as a powerful programming paradigm and outline the learning objectives for the lesson.	1
5-15 mins	What is OOP? Real-World Analogy	Explain the basic idea of OOP using analogies to real-world objects and their attributes and behaviours. Initiate a brief class discussion on examples.	2
15-25 mins	Core Concepts: Classes and Objects	Define and explain classes as blueprints and objects as instances. Emphasise the relationship between them using clear examples.	3
25-35 mins	Core Concepts: Encapsulation & Information Hiding	Explain encapsulation as bundling and information hiding as controlled access. Use analogies to illustrate the benefits of data protection.	4
35-45 mins	Core Concepts: Inheritance	Introduce inheritance as a mechanism for code reuse through superclasses and subclasses. Use hierarchical examples to demonstrate the concept.	5
45-55 mins	Core Concepts: Polymorphism	Explain polymorphism as "many forms" and the ability of objects to respond differently to the same method call. Use real-world scenarios for clarity.	6
55-60 mins	Advantages and Disadvantages of OOP & Wrap-up	Discuss the key advantages (modularity, reusability, etc.) and disadvantages (complexity, overhead, etc.) of using OOP. Briefly summarise the main concepts covered. Distribute the worksheet for the next activity.	7, 8

Teacher Notes

- **Abstract Concepts:** OOP concepts can be abstract for students new to programming. Use plenty of real-world analogies and visual aids to make them more concrete. Encourage students to come up with their own examples.

- **Distinguishing Concepts:** Students might struggle to differentiate between related concepts like classes and objects, or inheritance and polymorphism. Emphasise the "blueprint" nature of a class and the "instance" nature of an object. Use clear, contrasting examples for inheritance (an "is-a" relationship) and polymorphism (different responses to the same action).
- **Analogy Limitations:** Be aware that analogies have limitations. While helpful for initial understanding, ensure students eventually grasp the technical definitions.
- **Active Learning:** Encourage active participation through discussions and the worksheet activities. Pair or small group work can facilitate peer learning.
- **No Code Implementation:** Since the lesson is general and precedes language-specific details, focus on the conceptual understanding without getting bogged down in syntax.
- **Addressing Misconceptions:** Be prepared to address common misconceptions, such as thinking that OOP is always the best approach for every problem. Highlight that the choice of programming paradigm depends on the specific requirements of the project.
- **Time Management:** Be mindful of the time allocated for each section. The optional slides can be skipped if time is limited. Ensure sufficient time for the discussion on advantages and disadvantages, as this encourages critical thinking.
- **Worksheet Review:** Plan to briefly review the worksheet answers in a subsequent lesson to address any lingering misunderstandings. You could also use some of the student examples from the worksheet in future discussions.
